Linux System Audit: Cache & Memory Benchmark

User guide, result reference, command reference and glossary

LSA is a clean-room Linux implementation of AIDA64 Business's Cache & Memory Benchmark, built to produce results 1:1 with the native Windows app. AIDA64 is made by FinalWire Ltd.; LSA is an independent project, not affiliated with or endorsed by FinalWire, and contains none of its code or data.

Contents

1. Linux System Audit - User Guide 3
2. Linux System Audit - Result Reference 19
3. Linux System Audit - Command-Line Reference 26
4. Linux System Audit - Glossary 32

Linux System Audit - User Guide

Version 0.2.0. Covers Linux (Arch package, AUR, generic tarball, source build) and the Windows build.

> **About the name and credit.** Linux System Audit (LSA, package `lsa-sysinfo`) is a clean-room Linux implementation of AIDA64 Business features. **AIDA64 is made by FinalWire Ltd.** (https://www.aida64.com). LSA is an independent project. It is **not affiliated with, endorsed by or connected to FinalWire**. It contains no AIDA64 code, binaries or data. No AIDA64 code was disassembled, decompiled or copied, and AIDA64's hardware database and artwork are not used. Please do not send LSA questions to FinalWire. See `EVIDENCE.md` in the source for how the specification was made.

Other pages:

- RESULTS.md - one short section for every value LSA shows (each cell and line).
- CLI-REFERENCE.md - every command and option, with examples.
- GLOSSARY.md - every technical word used here, in plain English.

Contents

1. What it is, in plain words
2. Who it is for
3. Install
4. First launch
5. A tour of the window
6. What each number means
7. How the tests work (the detail)
8. Saving results and the run history
9. Reading results and comparing runs
10. Known limits and known differences from AIDA64
11. Windows build
12. Settings and files
13. Troubleshooting
14. Uninstall
15. FAQ
16. Glossary

---

1. What it is, in plain words

LSA measures **how fast your computer can move data** between the processor and its memory.

It does this for four places where data can live:

- **Memory** - your RAM sticks. Big, but the slowest.
- **L1 cache** - a tiny, very fast store inside each CPU core.
- **L2 cache** - bigger and a bit slower, also per core.
- **L3 cache** - bigger again, shared by many cores.

For each place it measures four things:

Test	Plain words	Unit
Read	How much data per second the CPU can read.	MB/s (memory) or GB/s (caches)
Write	How much data per second the CPU can write.	MB/s or GB/s
Copy	How much data per second it can read and write back somewhere else.	MB/s or GB/s
Latency	How long one single, unpredictable read takes. Lower is better.	ns (nanoseconds)

The result is a 4 x 4 grid, laid out like the familiar AIDA64 Cache & Memory Benchmark window, so you can compare a Linux run with a Windows run on the same PC.

This is the first tool of LSA. The project plans more of AIDA64's pages and tools later (`CHECKLIST.tsv` lists 162 planned items), but today LSA is the Cache & Memory Benchmark only.

2. Who it is for

- **Beginners** who want one number to check their RAM is running at the speed they paid for (for example after turning on XMP or EXPO in the BIOS).
- **Tuners and overclockers** who change memory timings, Infinity Fabric clock (FCLK) or similar BIOS settings and want to see the effect.
- **Dual-booters** who want to compare Linux with Windows, or LSA with AIDA64, on the same machine.
- **Developers and reviewers** who want a scriptable command-line benchmark with plain text output.

3. Install

LSA runs on 64-bit x86 Linux (Intel or AMD). It needs GTK 3 for the window. The command line needs only glibc.

3.1 Arch Linux and derivatives - AUR

The package name is **`lsa-sysinfo`**. With an AUR helper:

```
yay -S lsa-sysinfo
```

or by hand:

```
git clone https://aur.archlinux.org/lsa-sysinfo.git
cd lsa-sysinfo
makepkg -si
```

3.2 Arch Linux - prebuilt package from the release

Each GitHub release has `lsa-sysinfo-<version>-1-x86_64.pkg.tar.zst`. Install it with:

```
sudo pacman -U lsa-sysinfo-0.2.0-1-x86_64.pkg.tar.zst
```

3.3 Any other distribution - generic tarball

Each release also has `lsa-sysinfo-<version>-linux-x86_64.tar.gz`. It was built on an older distribution (Ubuntu 22.04), so it runs on most current ones (glibc 2.34 or newer, GTK 3). The fast AVX2 / AVX-512 code is picked at run time, only if your CPU has it.

```
tar xzf lsa-sysinfo-0.2.0-linux-x86_64.tar.gz
cd lsa-sysinfo-0.2.0-linux-x86_64
./usr/bin/lsa --version          # try it without installing
./usr/bin/lsa-gui
sudo cp -r usr/bin usr/share /usr/local/     # install for all users
```

Read `INSTALL.txt` inside the tarball for the full list of run-time needs.

3.4 Build from source

You need `gcc`, `make`, `pkg-config` and the GTK 3 development files (Arch: `gtk3`; Debian/Ubuntu: `libgtk-3-dev`; Fedora: `gtk3-devel`).

```
git clone https://github.com/eugenesflanagin-spec/lsa-sysinfo.git
cd lsa-sysinfo
make native                        # builds dist/native/lsa and dist/native/lsa-
gui
make install                       # per-user install into ~/.local
# or, system-wide (what the Arch package does):
sudo make install-system DESTDIR= SYSPREFIX=/usr
```

`make install` puts the programs in `~/.local/bin`. Make sure that folder is on your `PATH`.

3.5 Optional: memory details from srs-sysinfo

LSA can show your memory type and speed (for example "Dual Channel DDR5-6000 SDRAM (2 DIMMs)"). Reading that needs root, so LSA borrows it from a small cache file written by `srs-root`, part of the sister project **srs-sysinfo**:

```
yay -S srs-sysinfo        # or install it from its own release
pkexec srs-root            # one password prompt; writes ~/.cache/srs/root/
dmidecode.txt
```

Without it, the benchmark works fine. Only the "Memory Bus", "DRAM:FSB Ratio" and "Memory Type" lines stay empty.

3.6 Windows

See section 11.

4. First launch

Open your application menu and look for **LSA Cache & Memory Benchmark** (category System / Monitor; on some desktops under Benchmarks). Or type `lsa-gui` in a terminal.

The window opens with an **empty grid** and the system lines filled in. Nothing runs yet.



The window before a run, on the author's Ryzen 7 7800X3D. The grid fills in, cell by cell, when you press Start Benchmark.

Before you press **Start Benchmark**:

1. **Close heavy programs** - games, browsers with video, compiles, virtual machines, file copies. Anything that uses the CPU or memory makes the results lower and less steady.
2. **Never run another memory benchmark at the same time** - for example AIDA64 under Wine, or a second copy of LSA. Both runs would fight for the memory bus and both results would be wrong (latency can double).
3. Press **Start Benchmark**. A dialog warns that every CPU thread and the memory bus will be busy for about 20 seconds. Press **OK**.

The run takes about 20 seconds. The cell being measured turns yellow and shows `...`, and the status line says, for example, `Measuring L2 Cache Copy ...`. Your PC may feel slow during the run. That is normal.

To stop early, press the same button (it now says **Stop**). Closing the window also stops the run.

5. A tour of the window

There is one window. From top to bottom:

5.1 The results grid

	Read	Write	Copy	Latency
Memory	MB/s	MB/s	MB/s	ns
L1 Cache	GB/s	GB/s	GB/s	ns
L2 Cache	GB/s	GB/s	GB/s	ns
L3 Cache	GB/s	GB/s	GB/s	ns

Empty cells have not been measured yet. A cell that stays empty after a run was skipped (this can happen if you stopped the run).

5.2 The system lines

These describe the machine the numbers belong to. They are filled when the window opens and refreshed after each run (the clock is measured again).

Line	What it shows	Where it comes from
CPU Type	Core count as a word plus the model, e.g. `OctalCore AMD Ryzen 7 7800X3D`.	`/proc/cpuinfo`
CPU Stepping	Code name, stepping and family/model, e.g. `Raphael, stepping 2 (family 19h model 61h)`.	`/proc/cpuinfo` plus a built-in code-name table
CPU Clock	The core clock LSA **measured** a moment ago, e.g. `4968.7 MHz`. It moves a little run to run, because modern CPUs change clock all the time.	A short timing loop
CPU FSB	The reference (base) clock, **measured**, e.g. `105.0 MHz` on a board set to eCLK 105. Shows `100.0 MHz (nominal)` when it cannot be measured.	TSC rate x 100 / CPPC `nominal_freq`
CPU Multiplier	CPU Clock divided by CPU FSB, e.g. `46.16x`.	Calculated
North Bridge Clock	On AMD Ryzen 7000 desktop CPUs: the Infinity Fabric clock (FCLK), e.g. `2133.0 MHz`. Otherwise `n/a (...)` with the reason.	`ryzen_smu` driver's PM table (optional)
Memory Bus	The real memory clock, half the "MT/s" figure, e.g. `3000.0 MHz` for DDR5-6000.	srs-root DMI cache (optional)

Line	What it shows	Where it comes from
DRAM:FSB Ratio	Memory clock divided by CPU FSB, e.g. `DRAM:FSB 30:1`, or `28.6:1` with a 105 MHz FSB.	Calculated
Memory Type	Channels, type, speed and DIMM count, e.g. `Dual Channel DDR5-6000 SDRAM (2 DIMMs)`.	srs-root DMI cache (optional)
Chipset	The motherboard chipset and CPU platform, e.g. `AMD X870, AMD Raphael/Granite Ridge`.	PCI devices + hwdata `pci.ids`
Motherboard	Maker and board name.	`/sys/class/dmi/id/board_*`
BIOS Version	BIOS version and date.	`/sys/class/dmi/id/bios_*`

5.3 Status line and buttons

- **Status line** - before a run: `Ready - AVX-512 code path` (the fastest instruction set your CPU has, see 7.4). During a run: the cell being measured. After: `Finished - AVX-512 code path, 16 threads` (or `Stopped ...`). After saving: `Saved (PNG + TXT)`.
- **Save** - greyed out until a run finishes. Saves a picture and a text file (section 8).
- **Guide** - opens this user guide.
- **Start Benchmark / Stop** - starts a run (after asking), or stops the current one.
- **Close** - closes the window (and stops a run that is going).

The small grey note under the status line is the credit and non-affiliation statement.

5.4 Help for each value

Every cell and every system line has its own short section in the guide (RESULTS.md):

- **Right-click** a value and choose **Open guide: ...**, or
- point at it and press **F1**. (F1 anywhere else opens the guide at the start.)
- Hover over a value for a tooltip with its name.

The guide opens in your PDF viewer at that section if the viewer can jump to it (Okular, Evince, Papers); otherwise the HTML version opens in your web browser. On the command line, `lsa guide mem-latency` does the same; `lsa guide --list` lists the item names. The menu also has **LSA User Guide (PDF)**.

6. What each number means

6.1 Read, Write, Copy (bandwidth)

Bandwidth is how much data moves per second. Think of it as the width of a motorway: more lanes, more cars per second.

- **Read** - the CPU pulls data in. This is what most programs do most.

- **Write** - the CPU sends data out.
- **Copy** - read from one place and write to another. **LSA counts both the bytes read and the bytes written**, the usual convention for this kind of benchmark. So Copy is often close to Read + Write's combined traffic, not half of it.

Units:

- **Memory** is shown in **MB/s** (megabytes per second), e.g. `67678 MB/s`.
- **Caches** are shown in **GB/s** (gigabytes per second), e.g. `2292.3 GB/s`, because the numbers are huge.
- LSA uses decimal units: 1 MB = 1,000,000 bytes and 1 GB = 1,000 MB. The command line can show binary units instead (`--mib`).

All bandwidth tests use **every CPU thread at once**. So the number is the whole CPU's total, not one core's.

6.2 Latency

Latency is the waiting time for one piece of data. Think of it as how long one car takes to drive from A to B, no matter how wide the road is.

- It is measured in **nanoseconds** (ns). 1 ns = one billionth of a second.
- **Lower is better.**
- It uses **one thread** only, reading in a random order so the CPU cannot guess what comes next.

Typical values on a current desktop:

Level	Typical latency	Why
L1	about 1 ns (4-5 CPU clock ticks)	Inside the core, right next to it.
L2	about 3-4 ns	Still per core, a bit further away.
L3	about 10-12 ns	Shared by many cores.
Memory	about 60-90 ns (DDR5), 55-80 ns (good DDR4)	Off the CPU chip, through the memory controller to the RAM sticks.

6.3 Example: a full result

This is a run from the author's PC (Ryzen 7 7800X3D, 2 x 16 GB DDR5-6000), taken from the saved run history and shown in the command line's layout:

```
Linux System Audit 0.1.3 - Cache & Memory Benchmark

                    Read          Write           Copy      Latency
Memory          67678 MB/s     93211 MB/s     67373 MB/s      81.8 ns
L1 Cache       2292.3 GB/s    1279.0 GB/s    2459.7 GB/s       0.8 ns
L2 Cache       1240.2 GB/s    1198.0 GB/s    1215.6 GB/s       3.9 ns
L3 Cache       764.39 GB/s    637.68 GB/s    715.16 GB/s      11.6 ns
```

How to read it:

- The caches are 10 to 35 times faster than memory. That is why caches exist.
- L1 Write is about half of L1 Read. Each core can do two reads but only one write per clock tick.
- Memory latency (81.8 ns) is about 100 times L1 latency (0.8 ns).

7. How the tests work (the detail)

You do not need this to use LSA. It explains why the numbers look the way they do.

7.1 Bandwidth tests

- LSA starts **one worker thread per logical CPU** and pins each one to its own CPU ("pinned" = it is not allowed to move to another CPU).
- Each thread works on its **own block of memory**:
  - **L1, L2, L3:** the block is half of that thread's share of the cache, so it stays inside that cache level.
  - **Memory:** a block far bigger than all caches (at least 4 x L3, at least 256 MB, default 1 GB, at most a quarter of free RAM), so the data cannot sit in a cache.
- Each test runs many short trials of about 40-80 ms over about 1 second per cell, and keeps the **best** trial. Best, not average, because background activity can only make a trial slower.
- **Memory Copy** uses streaming stores (writes that bypass the cache), as is usual for memory-to-memory copies.

7.2 Memory Write on AMD CPUs (CLZERO)

On AMD Zen CPUs the link between each core complex and the memory controller can only accept about 16 bytes of normal writes per fabric clock. Normal "streaming" writes stop there (about 33 GB/s on a 7800X3D), well below what the RAM can absorb.

AMD CPUs have an instruction called **CLZERO** that writes a whole 64-byte line of zeros straight to memory. LSA uses it for **Memory Write** when the CPU has it: one thread per physical core, threads interleaved in 4 KB chunks over a shared block (256 MB or 2.5 x L3), best full pass. That measures what the memory itself can take, about 93,000 MB/s on the author's PC, which matches the reference.

- Other CPUs (Intel, older AMD) use streaming stores.
- To see the streaming-store figure on AMD, use `lsa cachemem --mem-write stream`.
- The command line's last line says which method was used.

7.3 Latency test

- One thread follows a **random chain of pointers** through every 64-byte line of a block. Each read tells it where the next read is, so the CPU cannot prefetch (fetch early) the data.
- **Block sizes:**
  - L1: half of the L1 data cache.
  - L2: half of the L2 cache.
  - L3: 8 x the L2 size (mostly L3 hits, still within the reach of the address-translation cache).

  - Memory: 8 x L3, at least 256 MB, at most half of free RAM. With 8 x L3 only about 1 in 8 reads can still hit L3.
- **Page size:** caches use normal 4 KB pages. Memory uses **2 MB huge pages** (Linux "transparent huge pages") so that address-translation misses do not inflate the figure.

7.4 Code paths (AVX-512, AVX2, SSE2)

LSA has three versions of its read/write/copy loops and picks the widest one your CPU supports:

Code path	Bytes per instruction	CPUs
AVX-512	64	AMD Zen 4 and newer, some Intel
AVX2	32	Most CPUs since about 2013-2015
SSE2	16	Every 64-bit x86 CPU

The status line and the last command-line line show which one was used. Cache bandwidth depends strongly on it; memory bandwidth hardly does. You can force a narrower one on the command line (`--isa avx2`) to compare.

8. Saving results and the run history

8.1 Save (window)

After a run, press **Save**. Pick a name (default `cachemem-YYYYMMDD-HHMMSS.png`). LSA writes **two files**:

- **`.png`** - a picture of the whole window, grid and system lines.
- **`.txt`** - the same results as text, in this layout:

```
Linux System Audit 0.1.3 - Cache & Memory Benchmark

                    Read          Write           Copy      Latency
Memory          67678 MB/s     93211 MB/s     67373 MB/s      81.8 ns
...

CPU Type            OctalCore AMD Ryzen 7 7800X3D
CPU Stepping        Raphael, stepping 2  (family 19h model 61h)
...
BIOS Version        1805  (09/18/2026)
Code path           AVX-512, 16 threads
```

8.2 Command-line output

`lsa cachemem` prints progress to the terminal's error stream (one line per finished cell) and the final grid plus the system lines to normal output. So you can save just the result with:

```
lsa cachemem > result.txt
```

8.3 The run history

Every run **finished in the window** (not stopped, and not command-line runs) adds one line to:

```
~/.cache/lsa/cachemem-history.txt
```

One line per run:

```
2026-10-21 10:47:23  AVX-512 16T 67678 93211 67373 81.8 2292344 1278983 2459687 0.8
1240159 1198037 1215578 3.9 764385 637682 715159 11.6
```

Part	Meaning
`2026-10-21 10:47:23`	Date and time of the run
`AVX-512`	Code path
`16T`	Threads used
next 4 numbers	Memory: Read, Write, Copy (MB/s), Latency (ns)
next 4	L1: Read, Write, Copy, Latency
next 4	L2: same order
last 4	L3: same order

Note: in the history **all bandwidth values are in MB/s**, caches too. `2292344` MB/s = 2292.3 GB/s. This makes the file easy to load into a spreadsheet (split on spaces).

9. Reading results and comparing runs

9.1 Rules for a fair comparison

1. **Same machine, same settings.** Change one thing at a time (one BIOS setting, one kernel).
2. **Idle machine.** Close other programs. Wait a minute after boot for background jobs to settle.
3. **Run 3 times** and look at the spread. On the author's PC the history shows L1 Read between 2178 and 2292 GB/s on normal runs. A difference smaller than the spread is noise.
4. **Look for outliers.** One run in the history has L1 Read 1903 GB/s and L3 Read 1171 GB/s, way off the others: something else was running. Throw such runs away.
5. **Compare like with like.** Do not compare an AVX-512 run with an AVX2 run, or a run with `--threads 8` with an all-thread run.

9.2 What changes what

You changed...	Expect to see change in...
Memory speed (MT/s), XMP/EXPO	Memory Read/Write/Copy up, Memory Latency down

You changed...	Expect to see change in...
Memory timings (CL, tRCD...)	Mostly Memory Latency
AMD FCLK / UCLK ratio (UCLK 1:1 vs 1:2)	Memory Latency and Memory Read/Copy
CPU clock (PBO, Curve Optimizer, power limits)	L1/L2/L3 bandwidth and latency in ns
Fewer DIMMs or single channel	Memory bandwidth roughly halves
Background load	Everything drops and gets noisy, L3 and Memory most

The author's own history shows one such change: Memory Latency went from 76-77 ns (block 4 x L3, LSA 0.1.2) to 80-82 ns (block 8 x L3, LSA 0.1.3). That was a **measurement** change, not a hardware change: with the bigger block fewer reads accidentally hit the L3 cache. Always note the LSA version when you compare old and new runs.

9.3 Comparing with AIDA64 on Windows

On the reference PC (Ryzen 7 7800X3D, DDR5-6000, FCLK 2000), three clean AIDA64 runs on Windows and three LSA 0.1.3 runs on Linux, same hardware, same day:

Cell	AIDA64 (3 runs)	LSA 0.1.3	Difference
Memory Read / Write / Copy	~63,700 / ~93,800 / ~63,600 MB/s	within ~3%	close
Memory Latency	80.2 / 81.5 / 82.8 ns	80.5 / 80.7 / 81.6 ns	close
L1 Read / Write / Copy	~2445 / ~1271 / ~2510 GB/s	Read 2250-2330, others within ~3%	**L1 Read ~5% low**
L1 Latency	0.8 ns	0.8 ns	same
L2 Read / Write / Copy	~1240 / ~1191 / ~1218 GB/s	within ~3%	close
L2 Latency	3.7 ns	3.6-3.9 ns	close
L3 Read / Write / Copy	~700 / ~714 / ~673 GB/s	within ~3%	close
L3 Latency	11.2 ns	11.6-11.7 ns	close

AIDA64 itself varies by a similar amount from run to run (for example L1 Read 2348-2457 GB/s, L2 Write 1014-1240 GB/s across runs).

10. Known limits and known differences from AIDA64

- **L1 Read is about 5% below AIDA64.** About 2250 GB/s vs 2445 GB/s on a 7800X3D. One core alone reaches about 61 bytes per clock tick, but with all cores loaded LSA gets about 289 GB/s per core against AIDA64's about 305. Longer runs (`--seconds 5`), one thread per core, 256-bit loads and per-thread buffer offsets were all tried; none closed the gap without pushing other cells away from the reference. So it is left as measured. See `CHANGELOG.md`.
- **Every other cell is within about 3%** of AIDA64 on the reference system.
- **Other CPUs are less tested.** The matching work was done on one AMD Zen 4 X3D system. Intel CPUs and other AMD generations work, but their numbers have not been compared side by side.

- **Memory Write on non-AMD CPUs** uses streaming stores, so it may read lower than AIDA64 on those.
- **North Bridge Clock** is read only on AMD Ryzen 7000 desktop CPUs (PM table version `0x540104`) with the `ryzen_smu` driver loaded; elsewhere it shows `n/a` and the reason. **CPU FSB** is measured on Linux when the CPU has an invariant TSC and ACPI CPPC; otherwise, and always on Windows, it shows `100.0 MHz (nominal)`.
- **Memory type and speed need the srs-root cache** (see 3.5).
- **Windows latency** is higher than Linux latency unless Windows grants large pages (see section 11).
- **Only the Cache & Memory Benchmark exists so far.** The other AIDA64 pages are planned.

11. Windows build

LSA also builds for 64-bit Windows, so you can run the same engine on the same PC under Windows and compare.

11.1 Install

Each release has two Windows files:

- **`lsa-sysinfo-<version>-setup.exe`** - installer. Installs for your user only, **no administrator rights needed**, into `%LOCALAPPDATA%\Programs\Linux System Audit`. Adds a Start-menu folder "Linux System Audit" with **LSA Cache & Memory Benchmark** and an uninstall shortcut. A desktop shortcut is optional (a tick box in the installer). It also appears in Settings -> Apps.
- **`lsa-sysinfo-<version>-win64-portable.zip`** - no install. Unzip anywhere and run.

Both contain:

- `lsa-gui.exe` - the Cache & Memory window (native Win32, no runtime needed).
- `lsa.exe` - the command line, same options as Linux (`lsa cachemem ...`).
- `LSA-User-Guide.html` - this guide (the installer adds a Start-menu shortcut to it). The **Guide** button and F1 in the window open it.

Windows SmartScreen may warn about an unknown publisher, because the files are not code-signed.

11.2 Differences on Windows

- **Save** writes a **BMP** picture plus a TXT file (Linux: PNG + TXT).
- The run history is `%LOCALAPPDATA%\LSA\cachemem-history.txt`.
- Memory details come from SMBIOS directly (no admin and no srs-root needed).
- CPU FSB shows `100.0 MHz (nominal)` and North Bridge Clock `n/a`: neither is read on Windows.
- **Latency and large pages:** the memory latency test uses 2 MB large pages only if your Windows account has the **"Lock pages in memory"** right. Without it LSA falls back to 4 KB pages, which adds address-translation time, so Memory Latency reads higher. The status line says which was used. To grant it: run `secpol.msc` -> Local Policies -> User Rights Assignment -> **Lock pages in memory** -> add your user, then sign out and in again. (`secpol.msc` exists on Pro / Enterprise editions.)

11.3 Uninstall on Windows

Settings -> Apps -> **Linux System Audit** -> Uninstall, or the uninstall shortcut in the Start menu folder. The portable zip: just delete the folder. The history file in `%LOCALAPPDATA%\LSA` stays; delete it by hand if you want.

12. Settings and files

LSA has **no settings file**. Everything is chosen per run (window: defaults; command line: options).

Path	What	Written by
`~/.cache/lsa/cachemem-history.txt`	One line per finished window run	`lsa-gui`
`~/.cache/srs/root/dmidecode.txt`	Memory details (read only)	`srs-root` from srs-sysinfo
your chosen `.png` + `.txt`	Saved results	Save button
`/usr/bin/lsa`, `/usr/bin/lsa-gui`	The programs	package
`/usr/share/applications/lsa-cachemem.desktop`	Menu entry	package
`/usr/share/applications/lsa-user-guide.desktop`	Menu entry "LSA User Guide (PDF)"	package
`/usr/share/doc/lsa-sysinfo/LSA-User-Guide.pdf`, `html/LSA-User-Guide.html`	This guide (PDF and HTML)	package
`/usr/share/doc/lsa-sysinfo/`	README, CHANGELOG, EVIDENCE, CHECKLIST, the guide's Markdown files	package
`/sys/kernel/ryzen_smu_drv/pm_table`	North Bridge Clock (read only, optional)	`ryzen_smu` driver

`lsa-gui` with arguments behaves as the command line (`lsa-gui cachemem ...` = `lsa cachemem ...`). `lsa-gui --gui-start` opens the window and starts a run straight away (it still asks first).

13. Troubleshooting

Problem	What to do
Numbers much lower than expected, or very different between runs	Something else is using the CPU or memory. Check with `top` or a system monitor. Close it, wait, run again. Laptops: plug in the charger and use a performance power profile.
Memory Latency is 120-160 ns, far above normal	Another memory benchmark (for example AIDA64 under Wine, or a second LSA) is running at the same time. Run only one.
	Install srs-sysinfo and run `pkexec srs-root` once (see 3.5). Restart the window.

Problem	What to do
Memory Bus / Memory Type say "run srs-root once" or "unknown"	
Memory speed shows lower than your kit's rating (e.g. 4800 instead of 6000)	Your BIOS is not using the XMP / EXPO profile. Enable it in the BIOS.
"could not allocate the test memory" (command line)	Not enough free RAM. Close programs and try again. (`--mem-mib` cannot go below 256 MiB or 4 x L3.)
Memory Latency higher than expected on Linux	Transparent huge pages may be off. Check `cat /sys/kernel/mm/transparent_hugepage/enabled`; it should show `[always]` or `[madvise]`, not `[never]`.
Status line says SSE2 or AVX2 on a CPU that has AVX-512	Some CPUs have AVX-512 turned off in the BIOS or by microcode. LSA uses what the CPU reports.
The window does not open from the terminal over SSH	It needs a desktop session. Use the command line `lsa cachemem` instead.
`lsa` runs a different program (prints a file list)	Your shell has an alias or function called `lsa` (some zsh and fish setups use it for `ls -a`). Run `command lsa cachemem`, use the full path `/usr/bin/lsa`, or remove the alias.
Windows: Memory Latency higher than on Linux	Grant "Lock pages in memory" (see 11.2).

14. Uninstall

- **Arch (AUR or package):** `sudo pacman -R lsa-sysinfo`
- **Generic tarball in /usr/local:** remove the files listed in the tarball, for example `sudo rm /usr/local/bin/lsa /usr/local/bin/lsa-gui /usr/local/share/applications/lsa-cachemem.desktop /usr/local/share/icons/hicolor/256x256/apps/lsa.png` and `sudo rm -r /usr/local/share/doc/lsa-sysinfo /usr/local/share/licenses/lsa-sysinfo`
- **`make install` (per user):** remove the same files under `~/.local`.
- **Your data:** `rm -r ~/.cache/lsa` deletes the run history. Saved PNG/TXT files are wherever you saved them.
- **Windows:** see 11.3.

15. FAQ

Is LSA the same as AIDA64? No. It is an independent, clean-room program that measures the same things in the same layout, so the numbers can be compared. It is not made by FinalWire and contains nothing from AIDA64.

Can it damage my PC? No more than any heavy program. It only reads and writes ordinary memory it allocated itself. It does not change any setting, clock or voltage. The run lasts about 20 seconds; the CPU gets warm like under any full load.

Why is Copy bigger than Read? Copy counts bytes read plus bytes written. For caches that adds up to more than Read alone.

Why are caches in GB/s and memory in MB/s? To match the familiar layout, and because cache figures would have too many digits in MB/s.

Why is my Memory Write much higher than Memory Read? On AMD CPUs with CLZERO, Memory Write measures the memory's own write speed (see 7.2). Reads have to come all the way back to the cores through the fabric, writes of zeros do not.

Does it need root? No. Only the optional memory details come from a root-made cache (srs-root).

Can I run it from a script? Yes: `lsa cachemem` prints plain text. It asks before it starts; add `--yes` in scripts (without a terminal and without `--yes` it does not start). See CLI-REFERENCE.md.

Why does the window say OctalCore? That is the core count written as a word (8 cores = Octal), as in the classic report style.

Does it work on ARM? No. The benchmark loops are x86-64 only.

16. Glossary

The full glossary is in GLOSSARY.md. The most important words:

- **Bandwidth** - how much data moves per second. Higher is better.
- **Latency** - how long one read takes. Lower is better.
- **Cache (L1, L2, L3)** - small, fast memory inside the CPU that keeps recently used data close.
- **MB/s, GB/s** - megabytes / gigabytes per second (decimal: 1 MB = 1,000,000 bytes).
- **ns** - nanosecond, one billionth of a second.
- **CLZERO** - an AMD instruction that writes a whole 64-byte line of zeros to memory.
- **AVX-512 / AVX2 / SSE2** - sets of wide CPU instructions that move 64 / 32 / 16 bytes at a time.
- **srs-root** - the root helper from srs-sysinfo that saves memory details LSA can read.

Linux System Audit - Result Reference

Back to the User Guide - Command-line reference - Glossary

One short section for every value LSA shows: the 16 cells of the grid and the lines below it. In the window, **right-click a value** and choose **Open guide**, or point at it and press **F1**, to jump straight here. On the command line, `lsa guide ITEM` does the same (`lsa guide --list` shows the items).

Each section starts with the plain-English meaning, then the detail. Example figures are from the author's PC (Ryzen 7 7800X3D, 2 x 16 GB DDR5-6000, FCLK 2133) unless a section says otherwise.

Memory Read

`mem-read` - grid cell, bandwidth

In plain words: how much data per second the whole CPU can read from your RAM sticks. Higher is better.

- Every CPU thread reads its own part of a block far bigger than all caches (default 1 GB in total, at least 4 x L3, at most a quarter of free RAM), so the data really comes from RAM.
- Shown in MB/s (1 MB = 1,000,000 bytes). Example: about 67,700 MB/s.
- What moves it: memory speed (MT/s), number of channels, on AMD the fabric clock (FCLK, see North Bridge Clock) and UCLK ratio, background load.
- Detail: How the tests work.

Memory Write

`mem-write` - grid cell, bandwidth

In plain words: how much data per second the CPU can write to RAM. Higher is better.

- On AMD CPUs with the CLZERO instruction LSA writes whole zeroed 64-byte lines straight to memory, one thread per physical core. That measures what the memory itself can take: about 93,000 MB/s on the example PC.
- Other CPUs use streaming stores (writes that bypass the caches). `--mem-write stream` shows that figure on AMD too; it is much lower there (about 33,000 MB/s), because of the link between the cores and the memory controller.
- Detail: Memory Write on AMD CPUs.

Memory Copy

`mem-copy` - grid cell, bandwidth

In plain words: how fast the CPU can read data from one place in RAM and write it to another.

- LSA counts **both** the bytes read and the bytes written, the usual convention, so Copy is close to Read rather than half of it. Example: about 67,400 MB/s.
- The writes are streaming stores. Same factors as Memory Read.

Memory Latency

`mem-latency` - grid cell, latency

In plain words: how long the CPU waits for one single, unpredictable read from RAM. Lower is better.

- One thread follows a random chain of pointers through a block of 8 x L3 (at least 256 MB, at most half of free RAM) on 2 MB pages, so the CPU cannot guess the next address.
- Shown in ns (nanoseconds). Example: about 80-82 ns. Typical DDR5: 60-90 ns.
- What moves it: memory timings (CL, tRCD, ...), memory speed, FCLK and the UCLK ratio.
- If it reads 120-160 ns, another memory benchmark is probably running at the same time.
- Detail: Latency test.

L1 Cache Read

`l1-read` - grid cell, bandwidth

In plain words: how fast all cores together read from their own tiny, fastest cache.

- Each thread works on half of its L1 data cache. Shown in GB/s. Example: about 2250-2330 GB/s.
- Depends on the CPU clock and the code path (AVX-512 reads 64 bytes per instruction, AVX2 32).
- Known difference: about 5% below AIDA64 on the reference system. See Known limits.

L1 Cache Write

`l1-write` - grid cell, bandwidth

In plain words: how fast all cores together write into their L1 cache.

- About half of L1 Read (example: about 1280 GB/s), because a core can do two reads but only one write per clock tick.

L1 Cache Copy

`l1-copy` - grid cell, bandwidth

In plain words: read and write inside the L1 cache at the same time.

- Counts bytes read plus bytes written, so it can be higher than Read. Example: about 2460 GB/s.

L1 Cache Latency

`l1-latency` - grid cell, latency

In plain words: the wait for one read that hits the L1 cache.

- One thread, random pointer chase over half of L1d. Example: 0.8 ns, about 4-5 clock ticks. It only changes with the CPU clock.

L2 Cache Read

`l2-read` - grid cell, bandwidth

In plain words: how fast all cores together read from their L2 caches (bigger and a bit slower than L1, still one per core).

- Each thread works on half of its share of L2. Example: about 1240 GB/s.
- Depends on the CPU clock.

L2 Cache Write

`l2-write` - grid cell, bandwidth

In plain words: how fast all cores together write into their L2 caches.

- Example: about 1200 GB/s. Depends on the CPU clock.

L2 Cache Copy

`l2-copy` - grid cell, bandwidth

In plain words: read and write inside the L2 cache at the same time (bytes read plus bytes written).

- Example: about 1215 GB/s.

L2 Cache Latency

`l2-latency` - grid cell, latency

In plain words: the wait for one read that hits the L2 cache.

- Random pointer chase over half of L2, 4 KB pages. Example: 3.6-3.9 ns.

L3 Cache Read

`l3-read` - grid cell, bandwidth

In plain words: how fast all cores together read from the L3 cache, the big cache the cores share.

- Each thread works on half of its share of L3. Example: about 700-760 GB/s.
- Depends on the CPU clock and, more than L1/L2, on background load (other programs use L3 too).

L3 Cache Write

`l3-write` - grid cell, bandwidth

In plain words: how fast all cores together write into the shared L3 cache.

- Example: about 640 GB/s.

L3 Cache Copy

`l3-copy` - grid cell, bandwidth

In plain words: read and write inside the L3 cache at the same time (bytes read plus bytes written).

- Example: about 715 GB/s.

L3 Cache Latency

`l3-latency` - grid cell, latency

In plain words: the wait for one read that hits the L3 cache.

- Random pointer chase over 8 x the L2 size. Example: about 11.6 ns.

CPU Type

`cpu-type` - system line

In plain words: which processor you have, with the core count written as a word.

- Example: `OctalCore AMD Ryzen 7 7800X3D` (8 cores = Octal). From `/proc/cpuinfo` (Windows: CPUID).

CPU Stepping

`cpu-stepping` - system line

In plain words: the CPU's design name and revision.

- Example: `Raphael, stepping 2 (family 19h model 61h)`. The code name comes from a built-in table of public CPU IDs; unknown CPUs show only family, model and stepping.

CPU Clock

`cpu-clock` - system line

In plain words: how fast one core was actually ticking a moment ago.

- LSA **measures** it with a short timing loop (it is not the number on the box). Example: about 4850-4970 MHz. It moves a little from run to run, because modern CPUs change clock all the time.

CPU FSB

`cpu-fsb` - system line

In plain words: the base (reference) clock that the CPU clock is a multiple of. Usually 100 MHz; some boards let you raise it (on ASUS AM5 boards this is "eCLK").

- **Linux:** measured as TSC rate x 100 / the CPU's nominal frequency (`/sys/devices/system/cpu/cpu0/acpi_cppc/nominal_freq`). The TSC is a counter that runs at the CPU's nominal clock times the base clock / 100, so this gives the real base clock. Example: `105.0 MHz` on a board set to eCLK 105.
- It needs an invariant TSC (`constant_tsc` in `/proc/cpuinfo`) and the ACPI CPPC `nominal_freq` file. Without them LSA shows `100.0 MHz (nominal)`.
- **Windows:** always `100.0 MHz (nominal)` - not measured.

CPU Multiplier

`cpu-multiplier` - system line

In plain words: how many times faster than the base clock the core runs.

- CPU Clock / CPU FSB. Example: 4846 MHz / 105.0 MHz = `46.16x`.
- When the FSB is the nominal 100 MHz, it is CPU Clock / 100.

North Bridge Clock

`nb-clock` - system line

In plain words: on AMD Ryzen, the speed of the Infinity Fabric (FCLK), the internal link between the cores and the memory controller. A faster fabric helps memory bandwidth and latency.

- Read from the PM table of the `ryzen_smu` kernel driver (`/sys/kernel/ryzen_smu_drv/pm_table`), read-only. Example: `2133.0 MHz` (FCLK 2133).
- Only the PM table layout `0x540104` (Ryzen 7000 desktop, Zen 4) is known to LSA. Other cases show why the value is missing:
  - `n/a (ryzen_smu driver not loaded)` - the driver is not installed or not loaded.
  - `n/a (PM table version 0x...... not supported)` - a CPU whose table layout LSA does not know.
  - `n/a (ryzen_smu pm_table not readable)` - the file exists but this user may not read it.
- **Windows:** not read (`n/a`).

Memory Bus

`memory-bus` - system line

In plain words: the real clock of your RAM, half the "MT/s" figure on the box.

- Example: `3000.0 MHz` for DDR5-6000.
- Linux: from the root DMI cache written by `srs-root` (srs-sysinfo), because reading it needs root. Without it the line says `run srs-root once for DIMM data`. Windows: from SMBIOS.

DRAM:FSB Ratio

`dram-fsb` - system line

In plain words: how many times faster than the base clock the memory clock is.

- Memory Bus / CPU FSB. A whole ratio is shown as `30:1`; otherwise with one decimal, for example `DRAM:FSB 28.6:1` (3000 MHz / 105.0 MHz).
- Needs the Memory Bus value (see above).

Memory Type

`memory-type` - system line

In plain words: the kind of RAM, its speed and how many sticks and channels.

- Example: `Dual Channel DDR5-6000 SDRAM (2 DIMMs)`. Same source as Memory Bus.
- If the speed is lower than your kit's rating, XMP / EXPO is probably off in the BIOS.

Chipset

`chipset` - system line

In plain words: the motherboard's support chip and the CPU platform.

- Example: `AMD X870, AMD Raphael/Granite Ridge`. The chipset model comes from the board name, the platform from the PCI root complex name in hwdata `pci.ids`.

Motherboard

`motherboard` - system line

In plain words: who made your motherboard and its model name.

- From `/sys/class/dmi/id/board_vendor` and `board_name` (Windows: SMBIOS).

BIOS Version

`bios` - system line

In plain words: the version and date of the firmware that starts the PC.

- From `/sys/class/dmi/id/bios_version` and `bios_date` (Windows: SMBIOS). Example: `1805 (09/18/2026)`.

Code path

`code-path` - status / last command-line line

In plain words: which set of CPU instructions the test loops used, and how many threads.

- AVX-512 (64 bytes per instruction), AVX2 (32) or SSE2 (16): the widest your CPU has, unless `--isa` asked for a narrower one. Cache bandwidth depends on it strongly, memory bandwidth hardly.
- The command line also lists the cache sizes LSA read from the kernel and the Memory Write method.
- Detail: Code paths.

Linux System Audit - Command-Line Reference

Back to the User Guide - Glossary

Linux System Audit is a clean-room Linux implementation of AIDA64 Business features. AIDA64 is by FinalWire Ltd.; LSA is not affiliated with or endorsed by FinalWire.

In plain words

There is one command, `lsa`. `lsa cachemem` runs the Cache & Memory Benchmark and prints the result grid as text; `lsa guide` opens this guide at the section for one value.

> **Before a run:** the benchmark loads every CPU thread and the memory bus for about 20 seconds. Like the window, the command line **asks first** (`Start now? [y/N]`). In scripts add `--yes`; without a terminal and without `--yes` it does not start. Close other heavy programs first, and never run it while another memory benchmark (for example AIDA64 under Wine) is running.

`lsa-gui` given any argument that does not start with `--gui` behaves exactly like `lsa` (for example `lsa-gui cachemem --only mem`). On Windows the same command is `lsa.exe`.

Synopsis

```
lsa cachemem [--threads N] [--seconds S] [--isa avx512|avx2|sse2] [--mem-mib M]
             [--only mem,l1,l2,l3] [--tests read,write,copy,latency] [--mib]
             [--mem-write auto|clzero|stream] [--yes]
lsa guide [ITEM] [--print]
lsa guide --list
lsa --version
lsa --help | -h
```

Commands

`lsa --help`, `lsa -h`

Prints the usage text. Exit code 0. (`lsa` with no arguments, or with an unknown command, prints the same text and exits with code 2.)

```
usage: lsa cachemem [--threads N] [--seconds S] [--isa avx512|avx2|sse2] [--mem-mib
M]
                    [--only mem,l1,l2,l3] [--tests read,write,copy,latency] [--mib]
                    [--mem-write auto|clzero|stream] [--yes]
       lsa guide [ITEM] [--print] | lsa guide --list
       lsa --version | --help

cachemem  Cache & Memory benchmark: Read / Write / Copy (MB/s) and Latency (ns) of
          Memory, L1, L2 and L3. Uses every CPU thread and the memory bus for ~20
s.
          Asks before it starts; --yes (or -y) starts at once, e.g. in scripts.
guide     Open the user guide at ITEM's section (e.g. mem-latency, cpu-fsb); --
print shows
          the command instead of running it; --list lists the items.
```

`lsa --version`

```
Linux System Audit (lsa) 0.2.0
Clean-room Linux implementation of AIDA64 Business features. AIDA64 is by FinalWire
Ltd. (aida64.com);
Linux System Audit is not affiliated with FinalWire. GPL-2.0-or-later.
```

`lsa cachemem [options]`

Runs the benchmark. With no options it measures all 16 cells (4 levels x 4 tests) with the defaults below, exactly as the window does. It asks first unless `--yes` is given. `lsa cachemem --help` prints the usage text and exits with code 0.

Options for `cachemem`

Option	Default	What it does
`--threads N`	all logical CPUs	Use only N worker threads for the bandwidth tests (on CPUs 0..N-1 of the allowed set). Values of 0, negative, or more than the CPU count mean "all". Latency always uses 1 thread.
`--seconds S`	`1`	Measuring time per cell, in seconds. Longer runs do not change the result much (5 s gives the same figures as 1 s on the reference PC); they mostly cost time.
`--isa avx512|avx2|sse2`	widest the CPU supports	Force a narrower code path. Asking for one the CPU lacks falls back to the best it has. Any other word stops with an error that lists the valid values.
`--mem-mib M`	auto (1024 MiB, capped at a quarter of available RAM)	Total size of the Memory bandwidth block, in MiB, for all threads together. Use a smaller value on PCs with little free RAM. Values below 256 MiB or 4 x L3 (whichever is bigger) are raised to that, so the block can never fit in the caches.

Option	Default	What it does
`--only LIST`	all levels	Measure only these levels. Comma-separated words: `mem`, `l1`, `l2`, `l3`.
`--tests LIST`	all tests	Measure only these tests. Comma-separated words: `read`, `write`, `copy`, `latency` (`lat` also works).
`--mib`	off	Show binary units: MiB/s for memory and GiB/s for caches (1 MiB = 1,048,576 bytes, 1 GiB = 1024 MiB).
`--yes`, `-y`	off	Start without asking. Needed when there is no terminal (scripts, cron). The environment variable `LSA_TEST_NOCONFIRM` does the same.
`--mem-write auto|clzero|stream`	`auto`	How Memory Write is measured. `auto`: CLZERO on AMD CPUs that have it, streaming stores elsewhere. `stream`: always streaming stores (shows the core-to-memory link limit on AMD). `clzero`: same as auto (CLZERO is used only if the CPU has it).

Notes:

- `--only` and `--tests` combine: `--only l1 --tests read` measures one cell (L1 Read). If you give only one of them, the other means "all".
- An unknown word in `--only`, `--tests`, `--isa` or `--mem-write` stops at once, before anything runs, with the valid values, for example: `cachemem: unknown --only value 'l4' (valid: mem, l1, l2, l3)` (exit code 2).
- An unknown option prints an error and the usage text and exits with code 2 without running anything.

Output

While it runs, one progress line per finished cell goes to **standard error**:

```
  L1 Cache Read      2292344 MB/s
```

(progress lines always show MB/s - MiB/s with `--mib` - and ns, without converting caches to GB/s).

At the end, the result goes to **standard output**:

```
Linux System Audit 0.1.3 - Cache & Memory Benchmark

                    Read          Write           Copy      Latency
Memory          67678 MB/s     93211 MB/s     67373 MB/s      81.8 ns
L1 Cache       2292.3 GB/s    1279.0 GB/s    2459.7 GB/s       0.8 ns
L2 Cache       1240.2 GB/s    1198.0 GB/s    1215.6 GB/s       3.9 ns
L3 Cache       764.39 GB/s    637.68 GB/s    715.16 GB/s      11.6 ns

CPU Type         OctalCore AMD Ryzen 7 7800X3D
CPU Stepping     Raphael, stepping 2  (family 19h model 61h)
CPU Clock        4968.7 MHz
CPU FSB          105.0 MHz  (multiplier 47.32x)
NB Clock         2133.0 MHz
```

```
Memory Bus       3000.0 MHz  DRAM:FSB 28.6:1
Memory Type      Dual Channel DDR5-6000 SDRAM  (2 DIMMs)
Chipset          AMD X870, AMD Raphael/Granite Ridge
Motherboard      ASUSTeK COMPUTER INC. TUF GAMING X870-PRO WIFI7 W NEO
BIOS Version     1805  (09/18/2026)
Code path        AVX-512, 16 threads; L1d 32 KB, L2 1024 KB, L3 98304 KB; Memory
Write: CLZERO, one thread per core
```

(Example values from the author's PC: the grid comes from a saved run, the system lines from the same machine with its base clock (eCLK) at 105 MHz and FCLK at 2133 MHz. What each line means: RESULTS.md.)

A one-cell run (`lsa cachemem --only l1 --tests latency --yes`, about 2 seconds) looks like this:

```
Linux System Audit 0.1.3 - Cache & Memory Benchmark

                     Read          Write           Copy      Latency
Memory                  -              -              -            -
L1 Cache                -              -              -       0.8 ns
L2 Cache                -              -              -            -
L3 Cache                -              -              -            -

CPU Type         OctalCore AMD Ryzen 7 7800X3D
...
Code path        AVX-512, 16 threads; L1d 32 KB, L2 1024 KB, L3 98304 KB
```

- A cell that was not measured shows `-`.
- The "Memory Write: ..." note at the end of the last line appears only when Memory Write was measured.
- Cache cells use two decimals below 1000 GB/s and one decimal above.
- The last line tells you the code path, thread count, the cache sizes LSA read from the kernel, and the Memory Write method.
- Command-line runs are **not** added to `~/.cache/lsa/cachemem-history.txt` (only window runs are).

Exit codes

Code	Meaning
0	Finished, or `--help` / `--version` / `guide`
1	The test memory could not be allocated (`cachemem: could not allocate the test memory`), or the guide could not be opened
2	No command, unknown command, unknown option or value, or the run was not confirmed

Examples

```
# Full run, save the result (asks first; the question goes to the terminal, not the
file)
lsa cachemem > cachemem-$(date +%F).txt

# Memory only (Read, Write, Copy, Latency) - about 6 seconds
lsa cachemem --only mem

# Just memory latency, e.g. after changing timings (one thread, about 1-2 seconds)
lsa cachemem --only mem --tests latency

# One cheap cell, to check the program works
lsa cachemem --only l1 --tests read

# Compare streaming-store Memory Write with the CLZERO figure (AMD)
lsa cachemem --only mem --tests write
lsa cachemem --only mem --tests write --mem-write stream

# Compare code paths
lsa cachemem --only l1,l2 --isa avx2

# A smaller memory block (e.g. a PC with little free RAM)
lsa cachemem --mem-mib 512

# Run 3 times and keep all results
for i in 1 2 3; do lsa cachemem --yes >> runs.txt; sleep 30; done
```

`lsa guide [ITEM] [--print]`

Opens the user guide at the section for one value, the same as right-clicking it in the window.

Command	What it does
`lsa guide`	Opens the guide at the start.
`lsa guide ITEM`	Opens the guide at ITEM's section, e.g. `lsa guide mem-latency`, `lsa guide cpu-fsb`.
`lsa guide ITEM --print`	Shows which section and which command would be used, without opening anything.
`lsa guide --list`	Lists the item names: `mem-read` … `l3-latency` for the 16 cells, `cpu-type`, `cpu-fsb`, `nb-clock` and so on for the lines below the grid, and `code-path`.

The guide opens in the PDF viewer if it can jump to a section (Okular, Evince, Papers), otherwise the HTML version opens in the web browser. It is looked for in `$LSA_DOC_DIR`, `/usr/share/doc/lsa-sysinfo` and beside the program. On Windows, `lsa.exe guide` opens `LSA-User-Guide.html` from the program's folder (at the start; Windows does not pass the section on).

```
$ lsa guide nb-clock --print
Item     nb-clock
Section  results--north-bridge-clock
Command  'okular' '/usr/share/doc/lsa-sysinfo/LSA-User-Guide.pdf#results--north-
bridge-clock'
```

Shell aliases named `lsa`

Some shell setups (Oh My Zsh, some fish configurations) define `lsa` as an alias for `ls -a`. If `lsa --version` prints a file list, run `command lsa ...` or `/usr/bin/lsa ...`, or remove the alias.

The window's command-line switches

Command	What it does
`lsa-gui`	Opens the Cache & Memory Benchmark window.
`lsa-gui --gui-start`	Opens the window and starts a run at once (the confirmation dialog still appears).
`lsa-gui cachemem ...`	Same as `lsa cachemem ...` (no window).

Linux System Audit - Glossary

Back to the User Guide - Command-line reference

Plain-English meanings of the words used in the LSA documents, A to Z.

- **AIDA64** - a system information and benchmark program for Windows made by FinalWire Ltd. LSA copies its Cache & Memory Benchmark layout so results can be compared. LSA is not affiliated with FinalWire.
- **AVX2** - "Advanced Vector Extensions 2". CPU instructions that handle 32 bytes at once. Found on most x86 CPUs since about 2013-2015.
- **AVX-512** - CPU instructions that handle 64 bytes at once. Found on AMD Zen 4 and newer and on some Intel CPUs. Gives the highest cache bandwidth figures.
- **Bandwidth** - how much data moves per second. Like the number of lanes on a road. Higher is better.
- **Best-of trials** - LSA repeats each measurement many times and keeps the fastest. Interruptions only ever make a trial slower, so the fastest one is the truest.
- **BIOS / UEFI** - the firmware that starts the PC and holds settings such as memory speed.
- **Bytes per clock** - how many bytes a core moves in one tick of its clock. A way to compare results at different clock speeds.
- **Cache** - small, very fast memory inside the CPU that keeps copies of recently used data, so the CPU does not have to wait for RAM.
- **Cache line** - the unit a cache works in: 64 bytes on current x86 CPUs. Reading one byte brings the whole 64-byte line in.
- **CCD / core complex** - on AMD Ryzen, a chip (die) holding up to 8 cores and their L3 cache.
- **Chipset** - the motherboard's support chip(s) that provide extra USB, SATA and PCIe connections.
- **CLZERO** - an AMD instruction that writes a whole 64-byte line of zeros to memory without first reading it. LSA uses it to measure Memory Write on AMD CPUs.
- **Code path** - which version of LSA's test loops is used: AVX-512, AVX2 or SSE2.
- **Copy** - reading data from one place and writing it to another. LSA counts both the bytes read and the bytes written.
- **DDR4 / DDR5** - the two current generations of desktop memory. DDR5 is faster in bandwidth; latency is similar.
- **DIMM** - one memory stick.
- **DMI / SMBIOS** - tables in the firmware that describe the PC (board, BIOS, memory sticks). Reading the memory part needs root on Linux.
- **dmidecode** - the Linux tool that reads DMI / SMBIOS tables. srs-root runs it and saves the output so LSA can read it without root.
- **DRAM** - "dynamic RAM", the kind of memory on your memory sticks.
- **DRAM:FSB ratio** - memory clock divided by the reference clock (FSB). DDR5-6000 runs its memory clock at 3000 MHz, so with a 100 MHz FSB the ratio is 30:1 (28.6:1 at 105 MHz).
- **Dual channel** - two independent paths between the CPU and memory. Twice the bandwidth of one. Usually means one stick in each of two channels.
- **eCLK** - on ASUS AM5 boards, the BIOS setting for the CPU's reference clock. Raising it from 100 MHz raises the CPU clock with it; LSA shows it as CPU FSB.

- **EXPO / XMP** - memory profiles stored on the sticks (EXPO by AMD, XMP by Intel). Turning one on in the BIOS runs the memory at its rated speed instead of a slow default.
- **Fabric / Infinity Fabric** - the internal link on AMD CPUs that joins the cores to the memory controller and the rest of the chip.
- **FCLK** - the Infinity Fabric clock on AMD CPUs. Higher FCLK usually lowers memory latency.
- **FSB** - "front side bus". In older PCs the bus between CPU and chipset; today used as the name for the reference clock (normally 100 MHz) that the CPU multiplier is applied to. LSA measures it on Linux from the TSC and the CPU's nominal frequency.
- **GB/s** - gigabytes per second. 1 GB/s = 1,000 MB/s in LSA.
- **GiB/s, MiB/s** - binary units: 1 MiB = 1,048,576 bytes. Shown with `--mib`.
- **Huge pages / transparent huge pages (THP)** - memory pages of 2 MB instead of 4 KB. Fewer pages means fewer address translations, so memory latency measures the memory, not the translation.
- **L1 cache** - the first, smallest and fastest cache, inside each core (32-48 KB for data on current CPUs). Latency about 1 ns.
- **L2 cache** - second level, per core, about 0.5-2 MB. Latency about 3-4 ns.
- **L3 cache** - third level, shared by many cores, 16-128 MB. Latency about 10-12 ns. "X3D" AMD CPUs have an extra-large stacked L3 (96 MB on a 7800X3D).
- **Latency** - the waiting time for one piece of data. Lower is better.
- **Lock pages in memory** - a Windows user right needed for large pages. Without it LSA's Windows build measures memory latency on 4 KB pages, which reads higher.
- **Logical CPU / thread** - what the operating system sees as a CPU. With SMT, each core shows as two logical CPUs.
- **MB/s** - megabytes per second. In LSA 1 MB = 1,000,000 bytes.
- **Memory controller** - the part of the CPU that talks to the memory sticks.
- **MT/s** - mega-transfers per second, the speed rating of memory (DDR5-6000 = 6000 MT/s). The actual memory clock is half of it.
- **Multiplier** - CPU clock divided by the reference clock (e.g. 46.16 x 105 MHz = 4847 MHz).
- **Nominal frequency (CPPC)** - the CPU's rated base clock as the firmware reports it through ACPI CPPC (`/sys/devices/system/cpu/cpu0/acpi_cppc/nominal_freq`), counted at a 100 MHz reference.
- **ns (nanosecond)** - one billionth of a second.
- **North Bridge clock** - on old PCs the chipset's clock; on current AMD CPUs the Infinity Fabric clock (FCLK). LSA reads it from the ryzen_smu PM table on Ryzen 7000 desktop CPUs.
- **Page / page walk** - memory is managed in pages (4 KB normally). Finding where a page really is takes a "page walk", which costs time if it is not cached.
- **Pinned thread** - a thread that is not allowed to move to another CPU. Keeps measurements steady.
- **PM table** - a block of live values (clocks, voltages, power) that the AMD SMU publishes. The `ryzen_smu` driver makes it readable at `/sys/kernel/ryzen_smu_drv/pm_table`. Its layout changes between CPU generations, so LSA only reads the layout it knows (version `0x540104`).
- **Pointer chase** - a chain where each value read is the address of the next read. The CPU cannot guess ahead, so each step shows the full latency.
- **Prefetch** - the CPU fetching data before it is asked for, because it guessed the pattern. Good for programs, bad for latency tests, so LSA uses a random chain.
- **RAM** - the main memory of the PC.
- **Read** - the CPU fetching data from a cache or memory.
- **ryzen_smu** - an optional Linux kernel driver that gives read access to the AMD SMU's PM table.

- **SMT** - "simultaneous multithreading" (Intel calls it Hyper-Threading). One core runs two threads.
- **SMU** - AMD's "System Management Unit", a small processor inside the CPU that controls clocks and power. LSA never sends it commands; it only reads the PM table through ryzen_smu.
- **srs-root** - the root helper from the sister project srs-sysinfo. Run once, it saves root-only hardware data (including memory details) into `~/.cache/srs/root`.
- **SSE2** - CPU instructions that handle 16 bytes at once. Every 64-bit x86 CPU has them.
- **Stepping** - the revision number of a CPU design.
- **Streaming store** - a write that goes straight to memory without being kept in the cache. Used for big copies so the cache is not filled with data that will not be read again.
- **TLB** - "translation lookaside buffer", a small cache of recent address translations. A TLB miss means a page walk.
- **TSC** - "time stamp counter", a CPU counter that ticks at a fixed rate (the nominal clock) on current CPUs (`constant_tsc`). LSA times it to measure the FSB.
- **UCLK** - on AMD CPUs, the memory controller clock. "1:1" means it runs at the memory clock; "1:2" means half. 1:1 gives lower latency when the memory can be run that way.
- **Write** - the CPU sending data to a cache or memory.
- **X3D** - AMD CPUs with an extra cache chip stacked on top ("3D V-Cache"), giving a very large L3.